

Temperatur- und Luftfeuchtigkeitsregulierer

In diesem Projekt wurde ein Regulierungssystem für Luftfeuchtigkeit und Temperatur entwickelt, das die Messdaten sicher speichert und für einen authentifizierten Nutzer die Messdaten der letzten Woche visualisiert. Zielwerte für die Temperatur sind zwischen 19 °C und 21 °C und für die Luftfeuchtigkeit zwischen 40 % und 55 %.

Das Projekt wurde bewältigt durch:

- Tim Dorozynski
- Lennart Esch
- Benjamin Hager
- Walter Neer
- Barnabas Steiner
- Jannis Weber

Inhaltsverzeichnis

Temperatur- und Luftfeuchtigkeitsregulierer.....	1
Anforderungen und Struktur.....	3
Phasen (patchnotes).....	5
Phase 1:.....	5
Phase 2:.....	7
Phase 3:.....	9
Phase 4:.....	12
Phase 5:.....	15
Phase 6:.....	17
Endspurt:.....	20
Projektstruktur.....	21
Deployment Diagramm.....	22
Services.....	23
Beschreibung der Komponenten.....	24
docker-compose.yml.....	24
CA.....	24
configurations.....	24
controller.....	24
environment.....	26
keycloak.....	26
mariadb.....	26
mosquitto.....	26
nginx.....	27
webapp.....	27
tests/E2E-tests.....	28
Installation und Inbetriebnahme.....	30
Pi Setup.....	30
WLAN AP.....	30
Pi Pico.....	31
Gehäuse.....	31
Backend server.....	32
Bedienung.....	32
E2E-tests Ergebnisse.....	33
Fazit.....	34
Anforderungen.....	34
Erfahrungsgewinn.....	36
Literatur und Hilfsmittel.....	38

Anforderungen und Struktur

Die Anforderungen für das Endprodukt waren die folgenden:

Funktionale Anforderungen:

Nr.	Funktion	Beschreibung	Prio
FA1	Sensormessung	Das System misst regelmäßig (z. B. alle 60 s) Temperatur und Luftfeuchtigkeit an der Klebestation.	Must
FA2	Datenübertragung	Messwerte werden vom Mikrocontroller sicher über WLAN/MQTT an den Backend-Server übertragen.	Must
FA3	Automatische Temperatursteuerung	Temperatur wird automatisch im Bereich 19–21 °C gehalten (Aktor: Heizung/Lüfter, Regelung per LSTM).	Must
FA4	Automatische Feuchtigkeitssteuerung	Luftfeuchtigkeit wird automatisch im Bereich 40–55 % gehalten.	Must
FA5	Web-Dashboard (Visualisierung)	Mitarbeiter sehen den zeitlichen Verlauf von Temperatur und Luftfeuchtigkeit grafisch im Leitstand.	Must
FA6	Manueller Eingriff	Bei Ausfall der automatischen Steuerung können Mitarbeiter manuell über das Dashboard Aktoren steuern.	Must
FA7	Datenspeicherung Kurzzeit	Sensordaten werden für mindestens 7 Tage online in einer Datenbank abrufbar gespeichert.	Should
FA8	Datenspeicherung Langzeit	Ältere Daten werden für mindestens 3 Jahre archiviert und können bei Bedarf wieder eingespielt werden.	Should

Nicht-funktionale Anforderungen:

Nr.	Anforderung	Beschreibung	Prio
NFA1	Sicherheit – Verschlüsselung	Alle Sensor- und Steuerungsdaten werden verschlüsselt (TLS) übertragen.	Must
NFA2	Authentifizierung & Autorisierung	Zugriff auf das Web-Dashboard nur mit gültigem Benutzerkonto und Passwort (z. B. Keycloak/IAM).	Must
NFA3	Netzwerkisolation	Das WLAN der Klebestation ist durch eine Firewall vom Unternehmens-LAN getrennt; eingehender Verkehr wird blockiert.	Must
NFA4	Verfügbarkeit / Zuverlässigkeit	Bei Ausfall der automatischen Steuerung muss manueller Eingriff weiterhin möglich sein (kein Single Point of Failure).	Must
NFA5	Kosteneffizienz	Gesamtkosten für Hardware und Software unter 10.000 €; Software soll Open Source sein.	Must
NFA6	Wartbarkeit	Betrieb mit standardisierten Docker-Containern für einfache Updates und Wartung.	Should
NFA7	Mobilität / WLAN	Kommunikation über WLAN, da die Klebestation häufig räumlich verschoben wird und Kabel nicht praktikabel sind.	Must
NFA8	Datenschutz (DSGVO)	Sensordaten sind keine personenbezogenen Daten — datenschutzrechtlich unkritisch, Compliance dennoch prüfen.	Should
NFA9	Benutzbarkeit	Das Dashboard muss für Leitstandmitarbeiter ohne IT-Kenntnisse intuitiv bedienbar sein.	Could

Zur Erarbeitung des Projekts wurde dieses in die folgenden 6 Phasen eingeteilt:

1. Planung und Einteilung der folgenden Phasen
2. Erstellen eines WLAN Access Points mit Firewall
3. Verdrahtung des Pico und Übertragen der Sensordaten an das Backend
4. Sicheres Speichern der Sensordaten in der Datenbank
5. Steuerung der Aktoren des Pico mittels LSTM-Netz
6. Erstellen einer sicheren Webapplikation für die Sensordaten

Phasen (patchnotes)

Die Präsentationen zu den einzelnen Phasen sind als PDF im jeweiligen Verzeichnis zu finden.

Im Folgenden sind die konkreten Änderungen während der einzelnen Phasen aufgelistet:

Phase 1:

Datum: 11.04.2026 - 23.04.2026

Problemstellung: Planung des Projektes und dessen 6 Phasen

Verantwortlicher: Jannis Weber

Projektplan:

☐	Aufgabe	Verantwor...	Status ⓘ	Fälligkeitsdat... ⓘ	Zeitleiste ⓘ	Abhängigkeit ⓘ
☐	▼ Phase 1: Project Pl... 5	JW	Erledigt	Apr. 28	Apr. 11 - 19	
☐	Unterelement	Owner	Status	Date	Zeitleiste ⓘ	Abhängig von ⓘ
☐	Create Feasibility Study	LE	Fertig	Apr. 16	Apr. 16 - 20	
☐	Create Requirement Elici...	JW	Fertig	Apr. 16	Apr. 16 - 18	
☐	Create Work breakdown ...		Fertig	Apr. 16	Apr. 16	
☐	Create GANT-Diagramm		Fertig	Apr. 17	-	Create Work breakdo...
☐	Create first Deployment ...		Fertig		Apr. 22 - 23	
☐	+ Unterelement hinzufügen					

Datei	Änderung	Erklärung
AnwendungInfoEli.pdf	N/A	Beschreibung der Anwendung und Projektidee mit Zielsetzung und Rahmenbedingungen
Machbarkeitstudie1.pdf	N/A	Machbarkeitsstudie zur technischen und wirtschaftlichen Umsetzbarkeit des Projektes
6Phasen.png	N/A	Erstmalige Verteilung der ersten Phasen

Machbarkeitsstudie:

Technik	Gegenstand der Bewertung	Bewertung
Technische Machbarkeit	Personal und Fachkompetenz; Hardware; Software	Vorkenntnisse in Web-Entwicklung und teilweise Elektrotechnik vorhanden; Hardware bereits gestellt; ausschließlich Open-Source-Software verwendbar
Wirtschaftliche Machbarkeit	Investitionskosten; Entwicklungskosten; Wartungskosten; Betriebskosten	Hardware bereits gestellt; kostenlose Software, keine Personalkosten; Betriebskosten beschränken sich auf Stromverbrauch und möglichen Hardware-Verschleiß
Organisatorische Machbarkeit	Integration in bestehende Systeme; Fachkenntnisse für Betrieb	Keine bestehende Systemlandschaft, keine Integration notwendig; Betrieb erfordert keine spezifischen Fachkenntnisse nach Demonstration
Zeitliche Machbarkeit	Zeit für Kompetenzgewinn; Zeit für Entwicklungsarbeit	Benötigte Kompetenzen können im Zeitrahmen erworben werden; ab dem 28.04.2026 ist die verfügbare Entwicklungszeit ausreichend
Rechtliche Machbarkeit	Personenbezogene Daten; Schutz erhobener Daten	Keine personenbezogenen Daten erhoben; Daten können durch Security-By-Design geschützt werden
Risiken und Gegenmaßnahmen	Frühere Abgabe; Verzug durch langsame Arbeit oder größeren Zeitaufwand bei Kompetenzaneignung	Risiko gering; Verzug wahrscheinlich, Gegenmaßnahme: Überstunden

Machbarkeitsbericht: Das Projekt wird unter den oben angegebenen Umständen als durchführbar eingestuft.

Plan für die nächsten Phasen:

To-do								
	Aufgabe	Verantwor...	Status	Fälligkeitsdat...	Zeitleiste	Abhängigkeit	Requirements	
☐	> Phase 1: Project Pl...	JW	Erfledigt	Apr-28	Apr. 11 - 19			
☐	> Phase 2: Deploy...		In Bearbeitung	Apr. 28	Apr. 20 - 28	Phase 1: Project Plann...		
☐	> Phase 3: Secure I...		In Bearbeitung	Mai 5	Apr. 27 - Mai...	Phase 2: Deploy...		
☐	Phase 4: Storing Cen...	LE	Start ausstehend	Mai 12	Mai 8 - 15	Phase 3: Secure In...		
☐	Phase 5: Controlling a...		Start ausstehend	Mai 20	Mai 15 - 23	Phase 4: Storing ...		
☐	Phase 6: Developing a...		Start ausstehend	Mai 27	Mai 23 - 30	Phase 5: Controlli...		
☐	Final: Present Everyth...	+5	Start ausstehend	Juni 10	Mai 27 - Juni...			
☐	+ Aufgabe hinzufügen							
				Apr. 28 - Jun...			2 Dokumente	

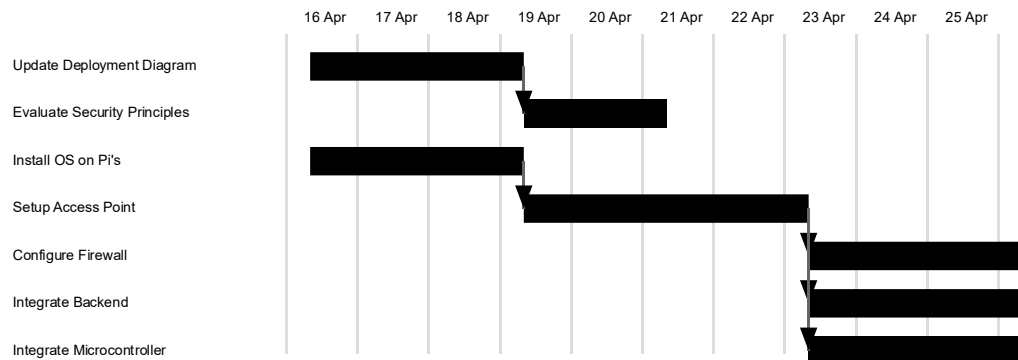
Phase 2:

Datum: 11.04.2026 - 28.04.2026

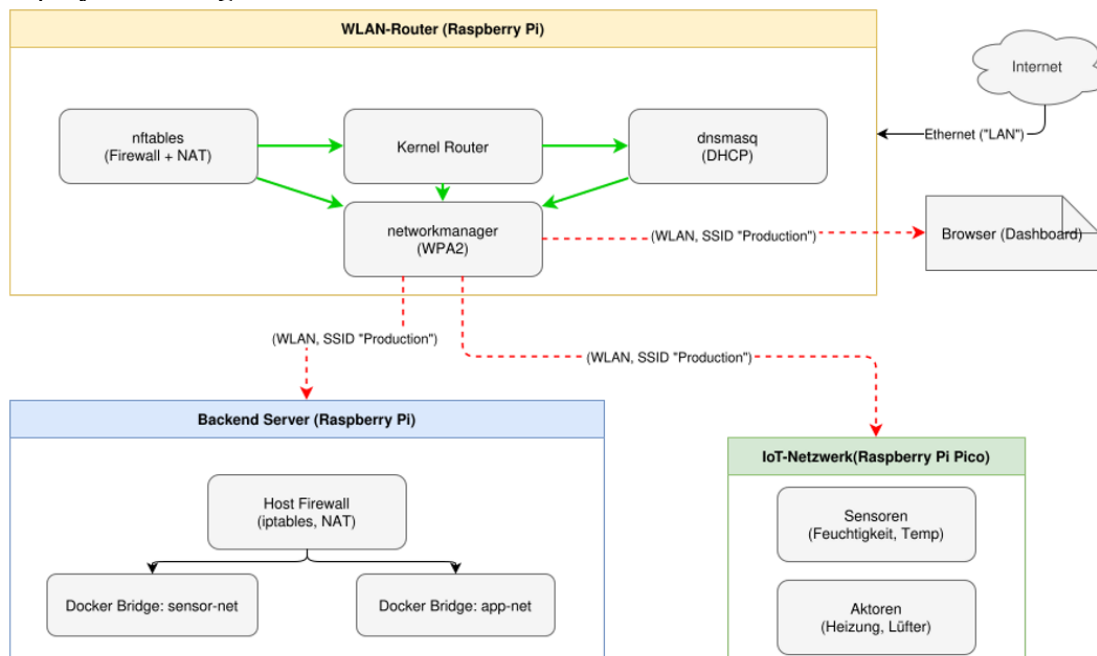
Problemstellung: Deployment der Kern-Infrastruktur

Verantwortlicher: Benjamin Hager

GANT-Diagramm:



Deployment-Diagramm:



Datei	Änderung	Erklärung
configuration s/wlan_ap_set up.py	Neu erstellt, danach mehrfach überarbeitet (Firewall-Regeln korrigiert, SSH auf allen Interfaces erlaubt, Quellen ergänzt)	Python-Skript zur Einrichtung eines WLAN-Access-Points auf dem Raspberry Pi: konfiguriert hostapd, dnsmasq, statische IP (192.168.4.1), IP-Forwarding und nftables-Firewall mit NAT (Masquerading über eth0). Ermöglicht das Produktionsnetzwerk "Production" für IoT-Clients.
configuration s/pico_main.py	Neu erstellt, danach erweitert (Phase 2 Integration)	MicroPython-Skript für den Raspberry Pi Pico W: verbindet sich als WLAN-Client mit dem Produktionsnetzwerk, ruft die Außentemperatur aus Friedrichshafen via Open-Meteo-API ab und steuert die Onboard-LED (Blinkrate abhängig von Temperatur: $<10^{\circ}\text{C} \rightarrow 2\text{s}$, $\leq 25^{\circ}\text{C} \rightarrow 1\text{s}$, $>25^{\circ}\text{C} \rightarrow 0.3\text{s}$).
pico_wlan_connection.txt	Neu erstellt, danach angepasst	Dokumentation/Notizen zur WLAN-Konfiguration des Pico W (SSID, Passwort, Verbindungsaufbau).

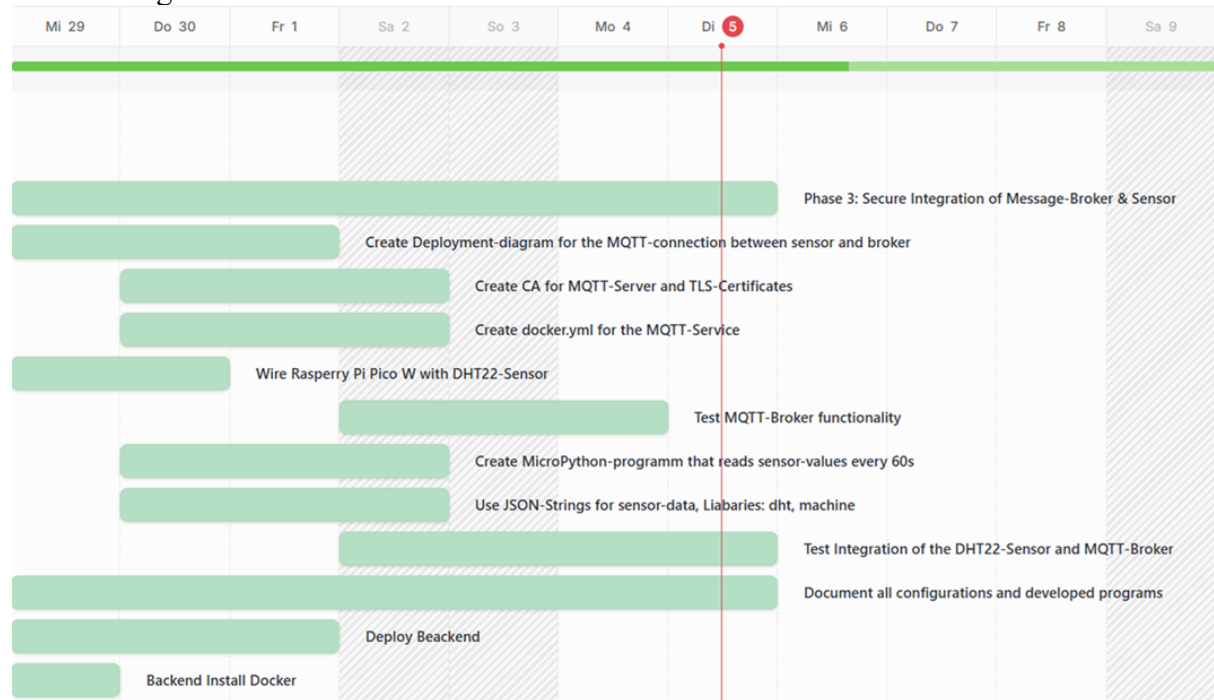
Phase 3:

Datum: 29.04.2026 - 05.05.2026

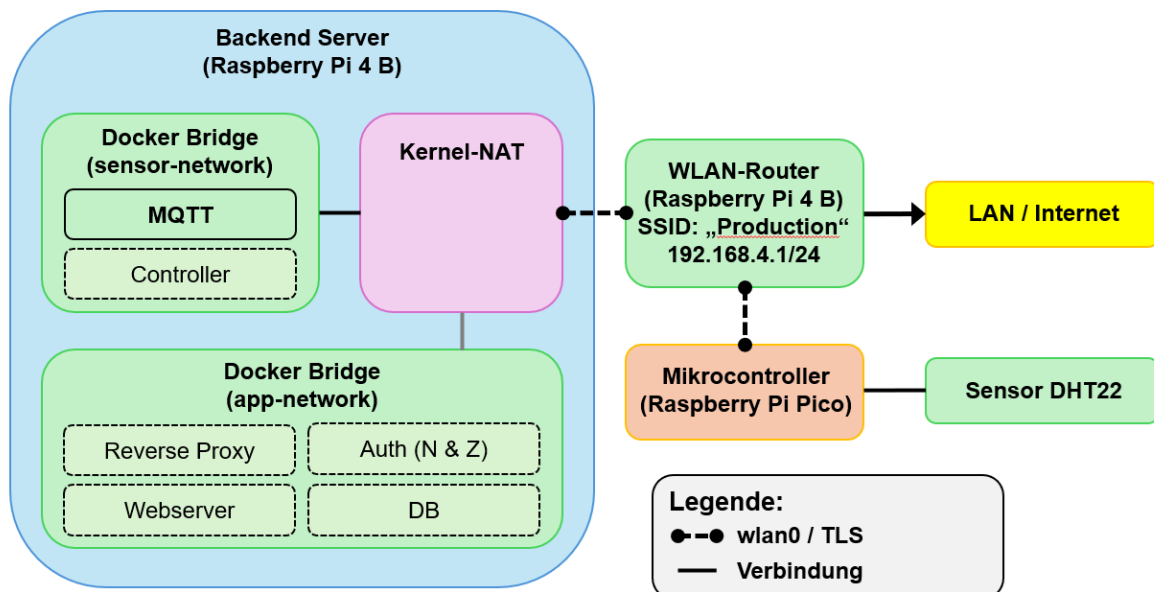
Problemstellung: Sichere Übertragung der Sensordaten an MQTT

Verantwortlicher: Walter neer

GANT-Diagramm:



Deployment-Diagramm:



Datei	Änderung	Erklärung
configurations/pico_main.py	In Module aufgeteilt	Aufgrund von Erweiterungen des Codes, für die Übersichtlichkeit in Module mit eigener Zuständigkeit aufgeteilt
configurations/pico/iot_config.py	Neu	Enthält alle Variablen für die Konfiguration von MQTT, CA und WLAN
configurations/pico/control.py	Neu	Initialisierung der Pico Pins und Zuweisung an Sensor und Aktoren
configurations/pico/wifi.py	Neu	Zuständig für Konfiguration und Aufbau einer WLAN-Verbindung
configurations/pico/mqtt.py	Neu	Zuständig für Konfiguration, Verarbeitung und Aufbau einer MQTT-Verbindung mit CA
configurations/pico/states.py	Neu	Steuert Aktoren je nach erhaltener Anweisung mit Zustandsinformation
configurations/pico/main.py	Neu	Hauptdatei des Pico, wird als erstes ausgeführt und ist für den Programmablauf verantwortlich
configurations/pico/lib/picozero	Schon in Phase 2 enthalten	Bibliothek für die Ansteuerung der On-Board-LED des Pico
configurations/pico/lib/umqtt	Neu	Bibliothek für MQTT relevante Funktionen, enthält simple.py und robust.py
configurations/pico/lib/lcd_api.py	Neu	Basisklasse für den Aufbau einer I2C-Verbindung
configurations/pico/lib/pico_i2c_lcd.py	Neu	Enthält Methoden für das Ansteuern eines LCD-Displays über den Pico, verwendet die Basisklasse aus lcd_api.py
CA/ca.key	Neu	Private Schlüsseldatei der CA, zum Signieren von Zertifikaten
CA/ca.srl	Neu	Seriennummerndatei der CA, enthält Seriennummern von signierten Zertifikaten
CA/ca.crt	Neu	Öffentliches Zertifikat der CA, zum Prüfen der Signatur anderer Zertifikate
CA/ca.der	Neu	Wie ca.crt aber als Binärdatei statt Text, damit der Pico damit arbeiten kann
CA/req.cnf	Neu	Konfigurationsdatei für OpenSSL mit Einstellungen für beispielsweise

Datei	Änderung	Erklärung
		Erweiterungen (weitere Hostnamen), wird für mqtt.csr und ca.crt verwendet
CA/mqtt.key	Neu	Privater Schlüssel des MQTT-Servers, wird für TLS-Verschlüsselung benutzt
CA/mqtt.csr	Neu	Wird aus mqtt.key erzeugt, für das Erhalten eines Zertifikats vom CA
CA/mqtt.crt	Neu	Signiertes Zertifikat des MQTT-Brokers, vom ca.key signiert, Beweis für Clients, dass das Zertifikat über CA gültig ist
docker-compose.yml	Neu, Konfiguration für mqtt-Container hinzugefügt	Konfiguration mit eclipse-mosquitto Image und Port 8883, benötigte volumes gemountet
mosquitto/config/mosquitto.conf	Neu	Konfigurationsdatei für den MQTT-Broker
mosquitto/log/mosquitto.log	Neu	Logdatei für den MQTT-Broker
mosquitto/secure/acc1	Neu	Angelegte Nutzer und deren Zugriffsrechte auf topics
mosquitto/secure/pwfile	Neu	Existierende Nutzer und deren verschlüsselte Zugangsdaten

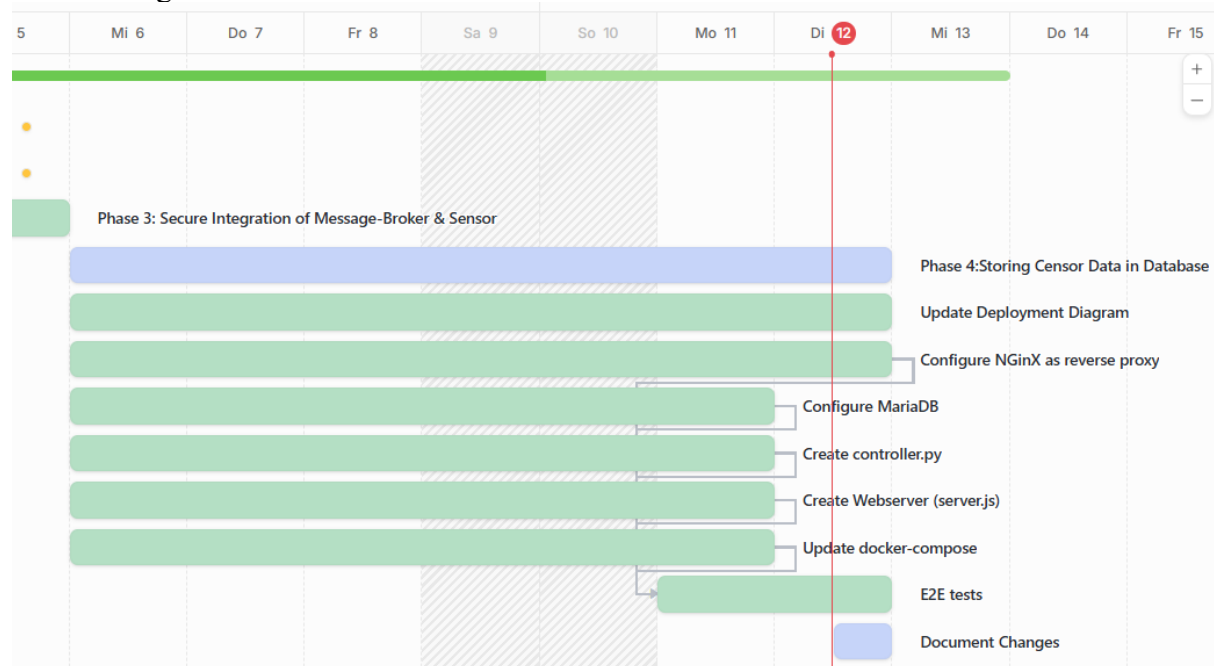
Phase 4:

Datum: 06.05.2026 - 12.05.2026

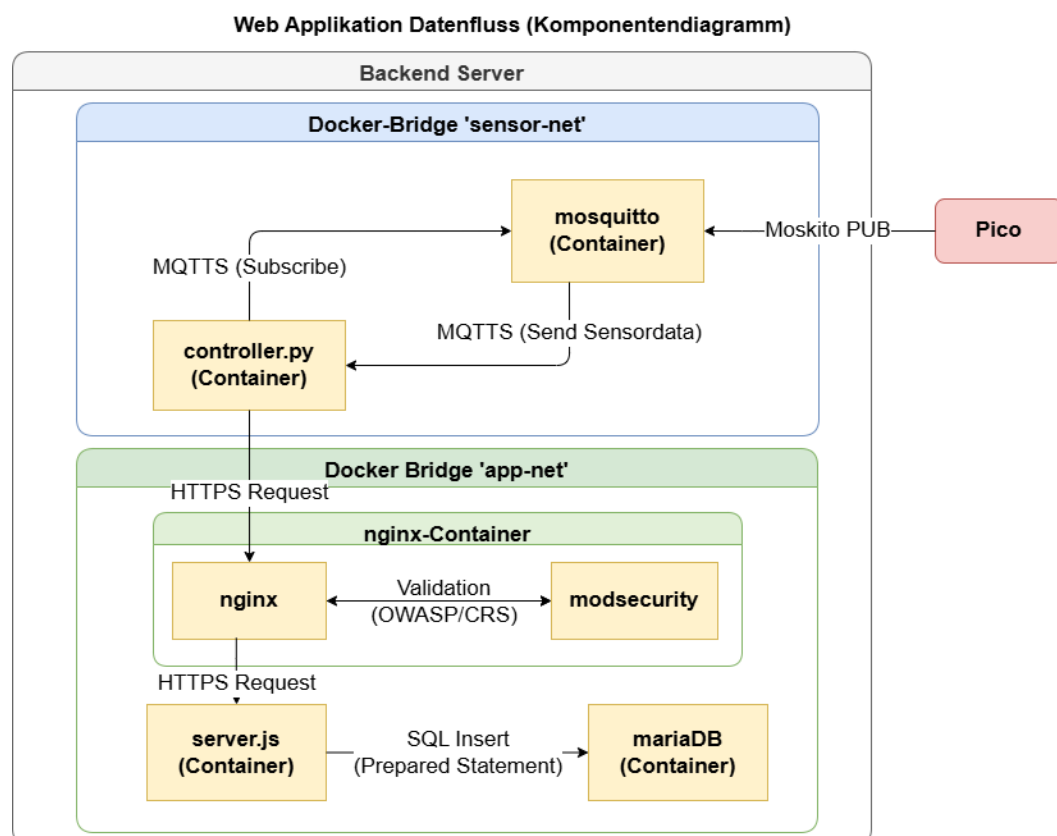
Problemstellung: Sichere Übermittlung der Sensordaten und Speicherung in der Datenbank

Verantwortlicher: Lennart Esch

GANT-Diagramm:



Deployment-Diagramm:



Datei	Änderung	Erklärung
controller/config.py	Neu	Liest Environment Variablen aus der Dockerfile
controller/controller.py	Neu	Main Datei des Controllers
controller/Dockerfile	Neu	Dockerfile, um aus dem Controller ein Image zu bauen, enthält die definierten Environment Variablen
controller/https_client.py	Neu	Verwendet die Request Library, um https requests zu schicken; wird verwendet für das Weiterleiten der Daten an nginx
controller/mqtt_handler.py	Neu	Verwendet die paho_mqtt Library um beim mqtt-Broker zu subscriben, definiert ein Event, das ausgelöst wird, wenn eine Nachricht vom Broker eingeht
controller/validation.py	Neu	Kontrolliert, ob die vom Broker erhaltene Nachricht valides JSON ist und prüft, ob die Werte plausibel sind (wird relevant, wenn der Pico angesteuert werden soll)
nginx/nginx.conf	Neu	Konfiguration von nginx, reverse Proxy von https://local.kleber.data zum Webapp Server und Redirect von http auf https
webapp/Dockerfile	Neu	Dockerfile zum Bau eines Images für den webserver
webapp/package.json	Neu	Node Standard-Konfigurationsdatei für den webserver
webapp/package-lock.json	Neu	Node Standard-Konfigurationsdatei für den webserver
webapp/server.js	Neu	Controller + Service-Layer für den webserver, enthält POST-Endpunkt zum Empfangen von neuen Messdaten und anlegen neuer Messwerte in der Datenbank
webapp/config/database.js	Neu	Erzeugt einen Connection-Pool für den Datenbank Zugriff, der von 'server.js' verwendet wird
mariadb/01_tables.sql	Neu	Initialscript für die Datenbank; legt die Tabellen für Messwerte und Messwertarchiv an
mariadb/02_archive_job.sql	Neu	Initialscript für die Datenbank; legt den Archivierungsjob an; es werden täglich alle

Datei	Änderung	Erklärung
		Daten, die älter als eine Woche sind von sensor-data zum sensor-data-archive verschoben
mariadb/03_user.sql	Neu	Initialscript für die Datenbank; legt einen Nutzer mit Schreibrechten für sensor-data an
docker-compose.yml	Konfiguration für die neuen Services hinzugefügt	Datenbank Konfiguration mit Health-Check und Verweis auf die Entry Point Skripte hinzugefügt Nginx Konfiguration basierend auf dem OWASP/modsecurity-crs Image hinzugefügt, enthält bereits Schutzmaßnahmen gegen Cross-Site-Scripting, SQL-Injektion und Directory Traversal Webapp Konfiguration basierend auf dem Image aus der Dockerfile hinzugefügt Controller Konfiguration basierend auf dem Image aus der Dockerfile hinzugefügt

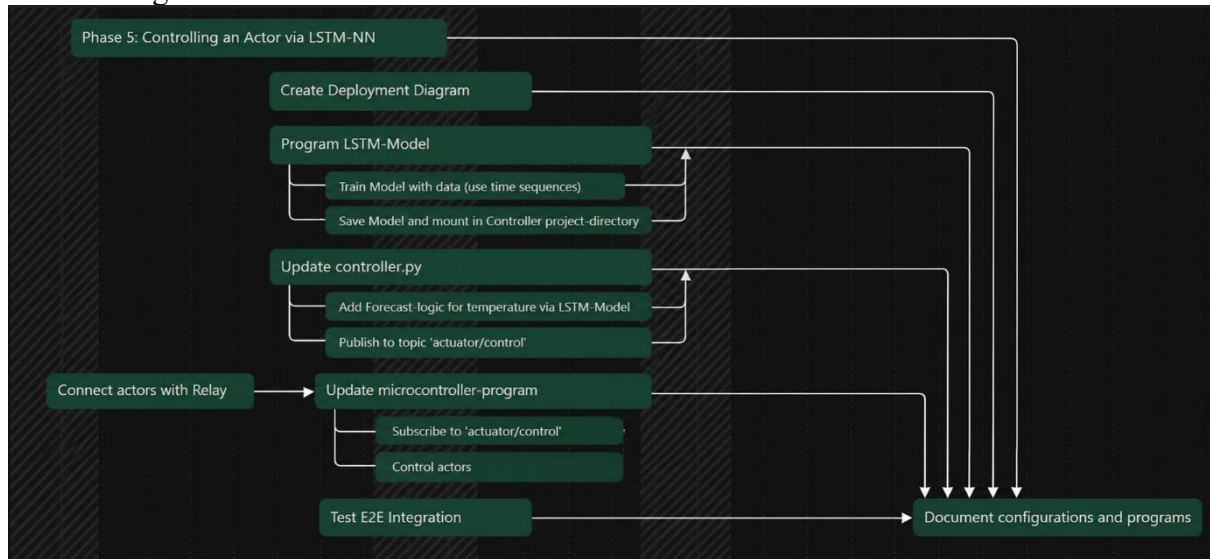
Phase 5:

Datum: 13.05.2026 - 21.05.2026

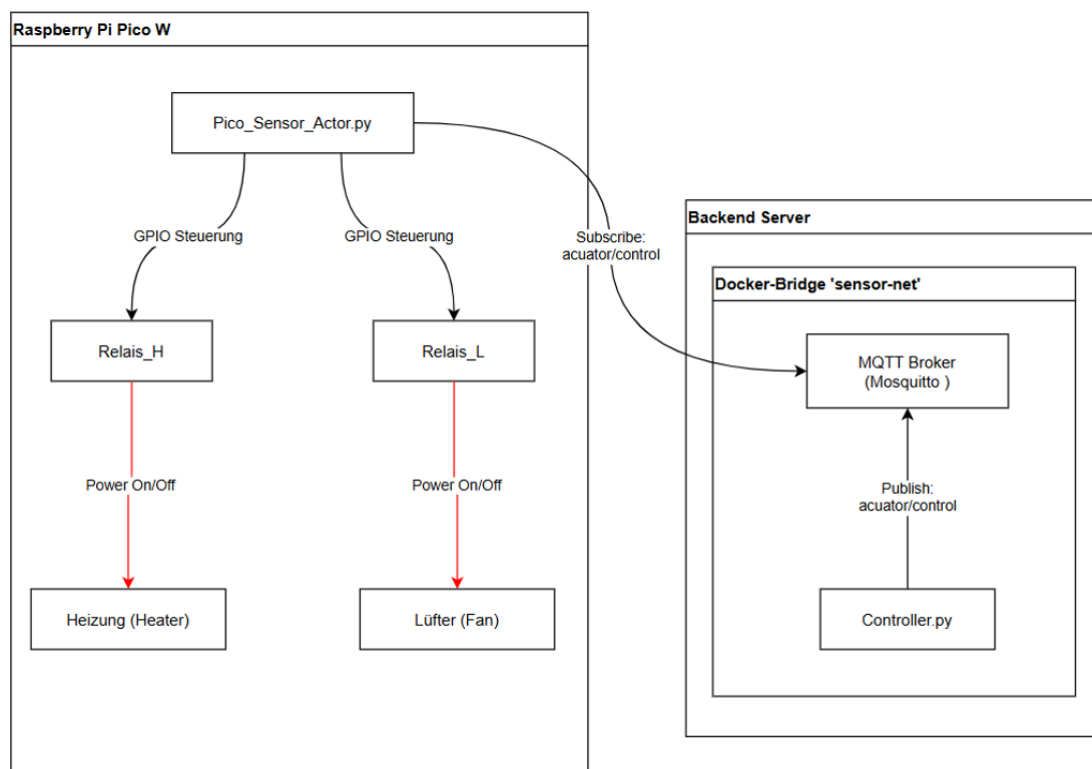
Problemstellung: Steuerung eines Aktors mittels LSTM-Neuronales Netz

Verantwortlicher: Tim Dorozynski

GANT-Diagramm:



Deployment-Diagramm:



Datei	Änderung	Erklärung
controller/lstm_handler.py	Neu	Lädt das trainierte LSTM-Modell und stellt Funktionen zur Verfügung, um Vorhersagen zu treffen; wird verwendet, um den optimalen Aktor-Wert basierend auf Sensor-Eingaben zu berechnen
controller/train.keras	Neu	Trainiertes Keras/TensorFlow LSTM-Modell; Gewichte und Architektur für die Vorhersage des optimalen Regelwertes
controller/train.py	Neu	Trainingsscript für das LSTM-Modell; verwendet historische Messdaten, um das Netz zu trainieren
controller/model_trainer.py	Neu	Modulare Trainingsfunktionen für das LSTM-Modell; ermöglicht automatisches Retraining basierend auf neuen Daten
controller/data_generation.py	Neu	Generiert Trainingsdaten aus Temperatur, Luftfeuchtigkeit, Heizung und Cooler; Daten werden algorithmisch generiert, um dem LSTM Ursache/Wirkung der Aktoren beizubringen
controller/lstm_weights.weights.h5	Neu	Exportierte Gewichte des trainierten LSTM-Modells; ermöglicht schnelleres Laden des Modells ohne Retraining
configurations/pico/control.py	Neu	Stellt Funktionen zur Ansteuerung des physischen Aktors (z.B. Heizung/Lüfter) bereit; implementiert PWM-Steuerung oder digitale Schaltausgänge
configurations/pico/states.py	Neu	Verwaltet die Zustände des Systems und des Aktors; definiert die möglichen Aktor-Positionen und deren Bedeutung
docker-compose.yml	LSTM-Umgebungsvariablen im Controller-Service hinzugefügt	Ermöglicht Konfiguration des LSTM-Modells und Trainingsparameter über Umgebungsvariablen; aktiviert LSTM-Vorhersagen im Controller
controller/Dockerfile	TensorFlow/Keras Dependencies hinzugefügt	Integriert erforderliche Python-Libraries (TensorFlow, keras, numpy, scikit-learn) für LSTM-Modellverarbeitung im Container

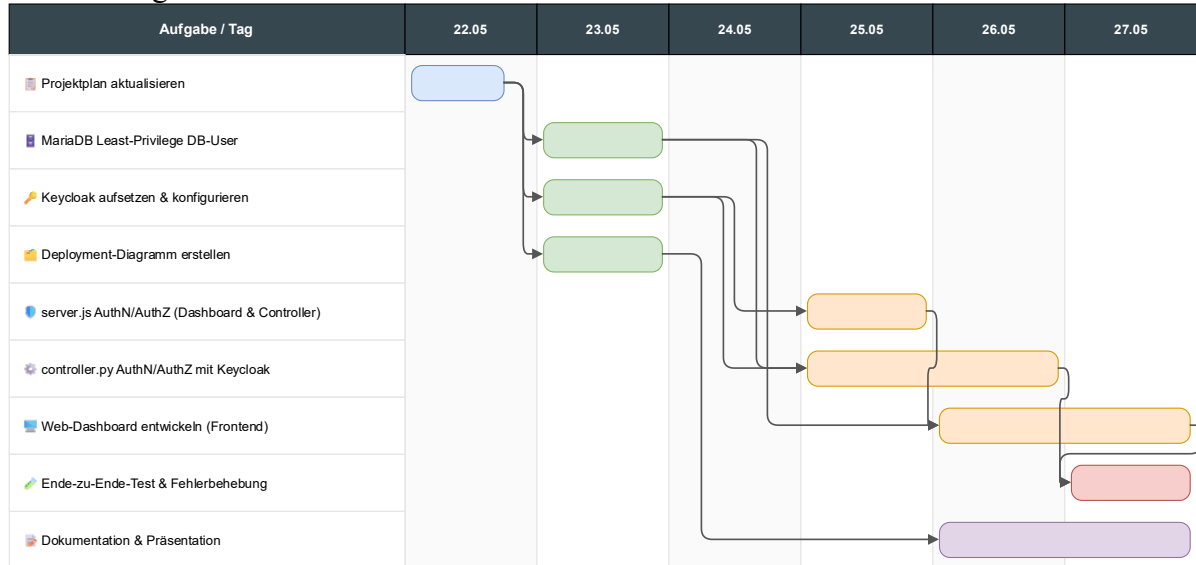
Phase 6:

Datum: 22.05.2026 - 28.05.2026

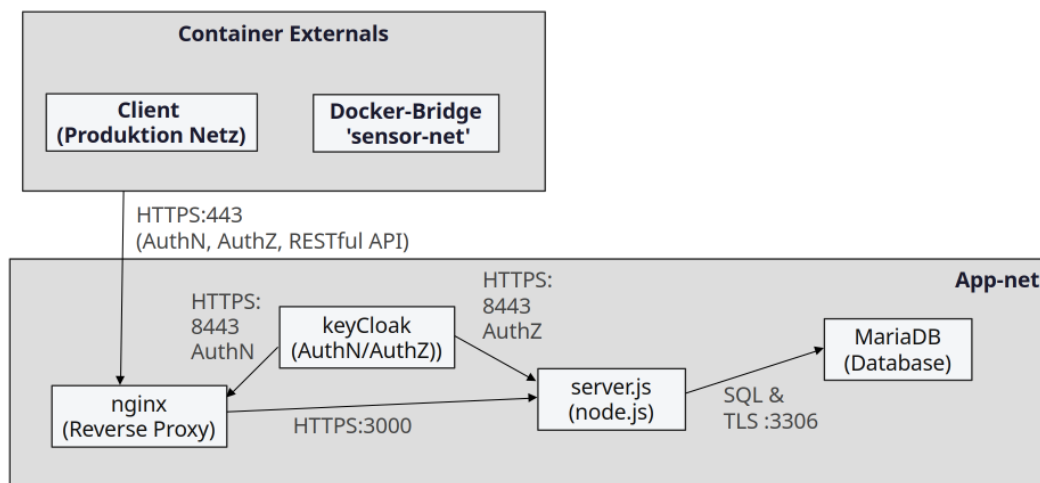
Problemstellung: Sichere Web-Applikation (AuthN/AuthZ mit Keycloak)

Verantwortlicher: Barnabas Steiner

GANT-Diagramm:



Deployment-Diagramm:



Datei	Änderung	Erklärung
docker-compose.yml	Service keycloak_web + Volume keycloak_data ergänzt, DB-Credentials in read/write getrennt, Health checks & depends_on	Integriert Keycloak als OIDC-Provider, hält dessen Daten persistent und sorgt für geordneten Startup der abhängigen Services
keycloak/iot-realm.json	Realm iot mit Rollen (dashboard-user, admin-user, controller-ingest), Benutzern und Clients angelegt	Definiert die Sicherheitsdomäne: dashboard-client (Browser, PKCE) und controller-client (Maschine, Client-Credentials)
nginx/nginx.conf	Route /auth auf Keycloak (Port 8443) + TLS-Zertifikatsprüfung, Trailing-Slash-Fix	Macht Keycloak über den Reverse Proxy erreichbar und verschleiert das interne System
mariadb/03_db_write_user.sql	Aus 03_user.sql umbenannt; Schreib-User webserv_write (INSERT, UPDATE)	Trennt schreibende DB-Zugriffe vom Lesen (Least Privilege)
mariadb/06_db_read_user.sql	Neuer Lese-User webserv_read mit SELECT auf sensor_data	Lese-APIs erhalten ausschließlich Leserechte
mariadb/05_training_data_privileges.sql	Privilegien an die Read/Write-Trennung angepasst	Konsistente Rechtevergabe nach der User-Aufteilung
webapp/config/database.js	getReadPool() / getWritePool() statt einem getPool()	Jeder Pool meldet sich mit eigenem DB-User an → Least Privilege auf Verbindungsebene
webapp/server.js	JWKS-Client + Middleware authenticateToken; neuer Endpoint GET /api/sensordata	Prüft das JWT (Signatur, Audience, Rolle dashboard-user); liefert Dashboard-Daten per JWT statt API-Key
webapp/package.json / package-lock.json	Dependencies jwks-rsa und jsonwebtoken ergänzt	Bibliotheken zur serverseitigen Token-Validierung
webapp/public/index.html	Keycloak-Login-Redirect + Einbindung lokaler keycloak.js	Leitet nicht angemeldete Nutzer zur Keycloak-Login-Seite weiter
webapp/public/keycloak.js	Neue lokale Kopie der Keycloak-JS-Bibliothek	Auslieferung über die eigene App statt über eine externe Quelle
webapp/public/frontend.js	Keycloak-Init + Sensordaten-Abruf per fetch() mit Bearer-Token	Initiiert AuthN/AuthZ und lädt Daten vom geschützten Endpoint

Datei	Änderung	Erklärung
webapp/public/chart.js	Diagramm-Rendering + Live-Update implementiert	Visualisiert Temperatur/Luftfeuchte, minütliche Aktualisierung
webapp/public/style.css	Styling-/Layout-Anpassungen	Optische Gestaltung des Dashboards
webapp/Dockerfile	Aus Root-Dockerfile nach webapp/ verschoben	Trennt den Web-App-Build vom Projekt-Root
sensor-net/keycloak_auth.py	Neu: Client-Credentials-Flow, Token-Cache, Rollenprüfung, Bearer-Header	Controller authentifiziert sich maschinell an Keycloak statt nur per API-Key
sensor-net/http_client.py	<code>_build_headers()</code> (API-Key + Bearer); <code>verify=False</code> → <code>verify=CA_CERT_PATH</code>	Zentrale Header-Erzeugung inkl. Token; echte TLS-Zertifikatsprüfung
sensor-net/controller.py	<code>verify_role()</code> vor <code>warmstart()</code> ; TLS-Verify aktiviert	Beendet den Controller, wenn die geforderte Keycloak-Rolle fehlt
sensor-net/config.py	KC_*-Variablen (Token-URL, Client-ID/-Secret, Rolle) + <code>CA_CERT_PATH</code> ergänzt	Konfigurierbare Keycloak-Parameter für den Controller
sensor-net/Dockerfile	ENV-Defaults für die Keycloak-Variablen	Standardwerte für den Containerbetrieb
sensor-net/mqtt_handler.py	Sendet Temperatur statt Prognose an <code>training_data</code>	Korrigiert den an die Trainingsdaten übermittelten Wert
configurations/wlan_ap_setup.py	Backend unter eigenem Hostnamen im Produktions-WLAN	Namensauflösung für den Zugriff auf das Backend

Endspurt:

Liste mit übrigen Aufgaben

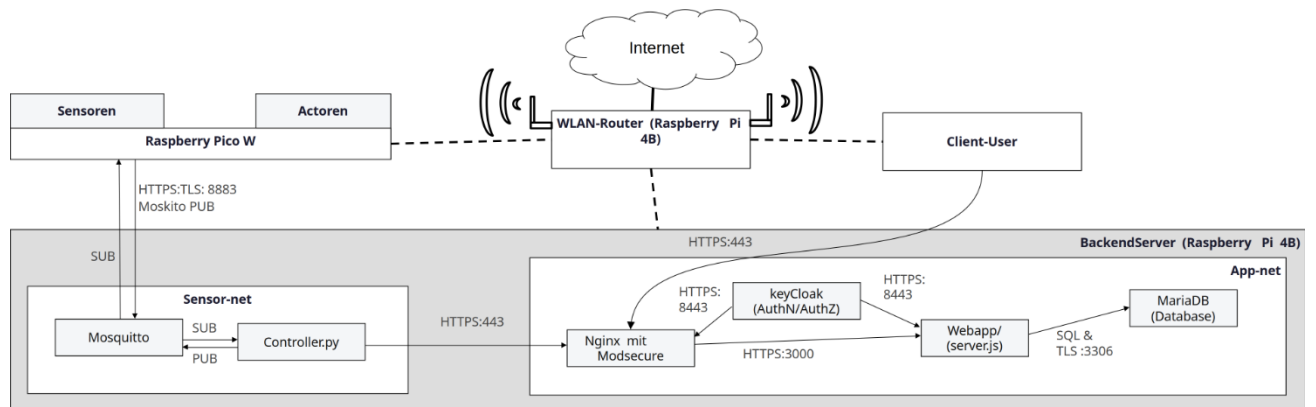
Datei	Änderung	Erklärung
CA/*	Neugennerierung aller Zertifikate; Alle Zertifikate sind jetzt unter CA abgelegt	Aufgrund von Schwierigkeiten bei den Zertifikaten wurden alle Zertifikate neu erstellt und befinden sich nun im CA-Verzeichnis
environment/*	Environment Variablen wurden von der docker-compose und den Dockerfiles in dieses Verzeichnis nach dem Namensschema [container_name].env ausgelagert	Die Environment Dateien sind jetzt an einem Ort gebündelt; Es war geplant die Applikation so zu gestalten, dass die Änderungen in den Environment Dateien reichen, um alle notwendigen Änderungen auf das gesamte Projekt auszuweiten; Aus Zeitdruck wurde das leider nicht mehr erreicht
docker-compose.yml	Anpassung an obere Änderungen; docker-compose stellt den Services die Zertifikate jetzt über volumes zur Verfügung	Die Zertifikate lagen vorher als Kopie im jeweiligen Verzeichnis, in dem sie gebraucht wurden; Jetzt werden die Zertifikate nur unter CA abgelegt und von der docker-compose als Volumes in die Container gemountet
controller/train.py	Entfernt	Wurde nicht mehr benötigt, weil die Trainingsmethode von model_trainer.py effizienter ist.
mariadb/05_db_admin_user.sql	Neu	Für Least Privilege wird ein neuer User benötigt, um die erweiterten Rechte des Admins abzubilden.
config/database.js & server.js	Dritter Connection-Pool getAdminPool() analog zu Read/Write hinzugefügt und neuer API Endpoint	Umsetzung von Least Privilege und erlaubt den Export beider Tabellen als CSV
keycloak/iot-realm.json	Hinzufügung Admin-user und Test-User	Nötige Identitäten für automatisierten Test und Admin Zugang

Projektstruktur

Erstellt mit MS-DOS `tree`

```
Anwendungsprojekt.Inf
  docker-compose.yml
  README.md
  CA
    (Zertifikate und Dateien zum anlegen)
  configurations
    wlan_ap_setup.py
    pico
      (Dateien des Pico)
  controller
    (Hauptdateien des Controller)
    weights
      (Gewichte für das LSTM-Netzwerk)
  documentation
    (Dokumentation)
  environment
    (Dateien für die Umgebungsvariablen der einzelnen Services)
  keycloak
    iot-realm.json
  mariadb
    (Initialisierungsskripte für die Datenbank)
  mosquitto
    config
      mosquitto.conf
    log
      mosquitto.log
    secure
      acl
      pwfile
  nginx
    nginx.conf
  webapp
    (Hauptdateien für den Webserver)
    public
      (Öffentliche Dateien für den Client)
  tests/E2E-tests
    (Testdateien für die Ende zu Ende Tests)
```

Deployment Diagramm



Services

Service	Abhängigkeiten	Netzwerk	Erklärung
mariadb / (db)	-	app-network	In mariadb werden die Daten gespeichert; Die Datenbank kommuniziert nur mit dem Webserver, welcher je nach Nutzer verschiedenen Zugriffsrechte hat; Der Start abhängiger Services wird verzögert, bis ein Health check absolviert wurde
keycloak / (keycloak_web)	-	app-network	Zuständig für Authentifizierung und Autorisierung der Nutzer des Webserver; Kommuniziert über nginx reverse-proxy mit dem Client und dem Webserver; Der Start abhängiger Services wird verzögert, bis ein Health check absolviert wurde
mosquitto / (mqtt)	-	sensor-network	Broker für die Kommunikation zwischen Pico und Controller; Kommunikation nach dem Publish/Subscribe-Prinzip; Auf Port 8883 nach außen sichtbar
node.js / (webapp)	db (healthy), keycloak_web (healthy)	app-network	Webserver für das Dashboard; Image selbstgebaut aus Dockerfile; Besitzt API-Endpunkte für den Datentransfer; Übergibt die öffentlichen Dateien an den Client
nginx / (owasp/modsecurity-crs:nginx)	Webapp	app-network	Schnittstelle für das app-network nach außen; Reagiert als reverse Proxy auf https-requests und leitet diese an Keycloak und webserver weiter; Es wird das owasp/modsecurity-crs Image verwendet als Schutz vor Cross-Site-Scripting, Sql-Injection und Directory Traversal
python / (controller)	mqtt, nginx	sensor-network	Controller für den Pico; Empfängt über mqtt Sensordaten und gibt diese über nginx an den webserver weiter; Erstellt aus den Messwerten mittels eines LSTM-Modells eine Messwertprognose und schickt über mqtt Steuersignale an den Pico

Beschreibung der Komponenten

docker-compose.yml

Hier werden die einzelnen Services definiert.

CA

CA steht für Certificate authority. In diesem Verzeichnis liegen alle verwendeten Zertifikate. Alle verwendeten Zertifikate sind selbstsigniert.

configurations

Unter configurations sind alle Dateien abgelegt, die nicht für das Backend benötigt werden. Hier liegen das Skript für den WLAN-Access-Point und unter pico die Dateien des Pico.

pico

Steuert Aktoren über Pins an, liefert Spannungsversorgung für den Sensor und liest Sensordaten aus.

Sendet Sensordaten über MQTT an den MQTT-Broker, liest vom controller gesendete Steuerbefehle über den MQTT-Broker.

wlan_ap_setup.py

Mit diesem Skript wird der WLAN-AP aufgesetzt.

Es wird mit `hostapd` ein Wifi aufgespannt.

Mit `dnsmasq` wird die Verteilung von IP-Adressen definiert. Das Backend erhält hierbei die statische IP `192.168.4.18` und den DNS-Namen `local.kleber.data`.

Mittels `nftables` wird eine Firewall erzeugt, die ungewollten Traffic automatisch blockiert.

controller

Empfängt Sensordaten vom MQTT-Broker, validiert und leitet sie per HTTPS an das Backend weiter.

Erstellt mithilfe eines LSTM-Modells Vorhersagen und publiziert Steuerungsbefehle (`COOL`, `HEAT`, `DRY`, `HUM`) an den Pico.

Authentifiziert sich per Keycloak Client Credentials Flow mit der Rolle `controller-ingest`.

config.py

Liest alle Betriebsparameter (MQTT, Backend-URL, Keycloak, Schwellenwerte) aus Environment-Variablen und konfiguriert das Logging.

controller.py

Einstiegspunkt des Controllers. Prüft beim Start die Keycloak-Authentifizierung und startet den MQTT-Loop.

keycloak_auth.py

Holt und cached das Keycloak Access-Token per Client Credentials Flow und prüft, ob die Rolle `controller-ingest` enthalten ist.

mqtt_handler.py

Baut den MQTT-Client auf und verarbeitet eingehende Nachrichten: Validierung, Weiterleitung ans Backend, LSTM-Vorhersage und Publish der Steuerungsbefehle auf `actuator/control`.

https_client.py

Leitet Sensordaten per HTTPS-POST an das Backend weiter, mit API-Key und Bearer-Token im Header.

validation.py

Prüft eingehende MQTT-Nachrichten auf gültiges JSON, vorhandene Felder und plausible Wertebereiche.

lstm_handler.py

Lädt das trainierte LSTM-Modell und erstellt Vorhersagen basierend auf den letzten 10 Messwerten.

model_trainer.py

Trainingsscript für das LSTM-Modell. Liest CSV-Daten, erstellt Sliding-Window-Sequenzen und trainiert das Modell.

data_generation.py

Generiert synthetische Trainingsdaten (30 Tage, Minutentakt) mit simulierter Thermostat-Logik.

environment

Enthält die Umgebungsvariablen für die Services. Es war geplant, dass durch eine Änderung an dieser Stelle alle benötigten Änderungen für alle Services automatisch durchgeführt werden, aber wegen Zeitknappheit konnte dies leider nicht mehr implementiert werden. Probleme waren besonders die Zertifikate, die durch ein Skript neu generiert werden müssten.

keycloak

Hier liegen die Daten des Identitäts- und Zugangsrechte-Verwaltungssystem (IAM).

- Rollen: admin-user, dashboard-user, controller-ingest
- Identitäten: testuser_norole, iotuser01, dashboard-user, admin, service-account-controller-client
- Clients: dashboard-client, controller-client

mariadb

Hier liegen die Entrypoint Skripte für den Datenbank-Container. In diesen Skripten werden:

- Tabellen für Sensordaten und ein Archiv angelegt
- Ein Job konfiguriert, der einmal täglich alle Messwerte, die älter als eine Woche sind ins Archiv verschiebt
- Nutzerrechte vergibt für einen iot-schreiber-, client-reader- und admin-export-Nutzer

mosquitto

Hier liegen die Konfigurationsdateien für den mosquitto-broker.

Unter config liegt die allgemeine Konfiguration, in der Port, Autorisierungsdateien und Zertifikate angegeben werden.

Bei log liegt eine leere log-file, die von mqtt nach dem Start verwendet wird.

Unter secure liegen die Anmeldedaten für mqtt.

nginx

Unter nginx liegt die entsprechende Konfigurationsdatei `nginx.conf`.

In dieser werden Private Key und öffentliches Zertifikat spezifiziert und mehrere reverse-proxys definiert.

Einmal bei https auf /auth zu Keycloak.

Dann bei https auf / zum Webserver.

Und auf http eine Umleitung auf https.

webapp

Unter webapp liegen alle Dateien für den webserver Container.

`config/database.js`

In dieser Datei sind Pools für die jeweiligen Datenbanknutzer definiert.

Hier speziell für den Read-, Write- und Admin-User.

public

Dieser Ordner beinhaltet alle Frontend-Elemente für das Anzeigen des Dashboards.

- **css** — Verwendete CSS-Elemente
- **external** — Extern importierte Skripte
- **img** — Bilddateien
- **js** — Verwendete Skripte
- **index.html** — HTML-Element für das Dashboard

`service/authentication.js`

Beinhaltet Middleware und Funktion für die Access Token Validierung

Speziell geprüft wird das Vorhandensein des Tokens und die erwarteten Audiences und Rollen.

Darüber hinaus ist der Endpoint für die Authentifizierung des API-Keys für die Datenbankzugriffe definiert.

`service/validateSensorPayload.js`

Beinhaltet Funktion um Übertragungsinhalt der Sensoren (Temperatur, Luftfeuchtigkeit, Zeitstempel) zu validieren.

Es wird speziell geprüft:

1. Sind Temperatur und Luftfeuchtigkeit als Zahlen vertreten
2. Ist das Format des Zeitstempels valide
3. Sind Temperatur und Luftfeuchtigkeit innerhalb des definierten Bereiches
4. Ist der Zeitstempel aktuell

Damit werden korrekte Einträge in der Datenbank gewährleistet.

`server.js`

Hauptkomponente zum Starten des Webservers. Beinhaltet ebenfalls API für Lese- und Schreibzugriffe auf die Datenbank.

Gegebene Endpoints:

- **GET /api/status** — Serverstatus abfragen
- **GET /api/sensordata** — Aktuelle Sensordaten erhalten
- **GET /api/sensordata/range** — Sensordaten von einem bestimmten Zeitfenster erhalten
- **GET /api/admin/export** — Aktuelle und archivierte Sensordaten als CSV erhalten
- **POST /api/internal/sensordata** — Sensordaten in Datenbank schreiben

tests/E2E-tests

End-To-End-Tests, die die gesamte Pipeline vom MQTT-Publish bis zur Datenbank bzw. zum Dashboard abdecken.

Test 1: Sensordaten → Datenbank (Happy Path)

```
MQTT-Nachricht publizieren → Broker → controller.py → HTTP POST an Backend → Wert in MariaDB prüfen
```

- Simulierte MQTT-Nachricht `{"temperature": 17.77, "humidity": 44.33}` auf `sensor/data` publishen
- Warten bis der Wert durch die Pipeline fließt
- Prüfen: Ist der Wert im Zeitfenster über `/api/sensordata/range` auffindbar?

Test 2: Keycloak AuthN/AuthZ (Token-Flow)

```
Controller holt Token → JWT dekodieren → Rolle prüfen → Request mit Bearer Header → Backend akzeptiert
```

- Client Credentials Flow: Token von Keycloak mit `controller-client` holen
- Prüfen: Hat das JWT die Rolle `controller-ingest` in `realm_access.roles`?
- Prüfen: Akzeptiert `/api/sensordata` einen Request nur mit Bearer Token?

Test 3: Unauthentifizierte Requests werden abgelehnt

```
Request ohne Auth → nginx → server.js → 401/403 Rejected
```

- POST ohne jegliche Header → erwartet 401
- POST mit falschem API-Key → erwartet 401
- GET ohne Bearer Token → erwartet 401
- GET mit ungültigem Bearer Token → erwartet 403

Test 4: Ungültige Sensordaten werden verworfen

```
Ungültige MQTT-Nachricht → Broker → controller.py (verwirft) → kein neuer Eintrag in DB
```

- 5 verschiedene ungültige Payloads publishen (fehlende Felder, falscher Typ, leerer String, kein JSON, Extremwerte)
- Warten, bis Pipeline verarbeitet hätte
- Prüfen: kein neuer Eintrag in der Datenbank seit Testbeginn

Test 5: Aktor-Steuerung bei hoher Temperatur

```
MQTT "sensor/data" (35°C) → controller.py → LSTM-Modell → MQTT "actuator/control" = "COOL"
```

- 15 Nachrichten mit `temperature: 35.0` publishen um den LSTM-Buffer zu füllen
- Gleichzeitig auf `actuator/control` Topic subscriben
- Prüfen: Kommt ein `COOL`-Befehl vom Controller zurück?

Test 6: TLS-Zertifikatsvalidierung

```
HTTPS-Request → nginx (mit CA-Zertifikat validiert)
```

- Prüfen: CA-Zertifikatsdatei existiert auf dem System
- Prüfen: HTTPS-Verbindung mit korrektem CA-Zertifikat funktioniert
- Prüfen: HTTPS-Verbindung mit falschem Zertifikat wird abgelehnt (SSLError)

Test 7: Dashboard-Zugriff (Rollenbasiert)

```
User → Keycloak (Login + Token) → nginx → server.js → Rolle prüfen  
→ Zugriff erlauben/verweigern
```

- 7a: Admin-User (admin-user + dashboard-user) → 200 auf /api/sensordata + 200 auf /api/admin/export
- 7b: Normaler User (dashboard-user) → 200 auf /api/sensordata + 403 auf /api/admin/export
- 7c: User ohne Rolle (testuser_norole) → 403 auf /api/sensordata

Installation und Inbetriebnahme

Pi Setup

- Installiere das Raspberry Pi light Image auf beiden Pis
- Lies die MAC-Adresse eines Pis aus; dieser Pi wird später zum Backend

WLAN AP

- Verbinde den WLAN Pi über ein Netzkabel mit dem Internet
- Stelle eine Verbindung zum Pi via ssh her
- Setze backend_mac_address in Zeile 4 in /configuration/wlan_ap_setup.py auf die MAC-Adresse des Backends
- Führe sudo apt install hostapd dnsmasq auf dem Pi aus
- Kopiere jetzt das Skript wlan_ap_setup.py auf den Pi und führe es mit root-Rechten aus

Pi Pico

- Verbinde den Pi Pico über Micro-USB an USB-A/USB-C eines Endgerätes mit Thonny:
 1. BOOTSEL-Taste (weiße Taste auf Pico) gedrückt halten
 2. An Entwicklungs-Endgerät per USB-Kabel anschließen
 3. .uf2-Datei mit der MicroPython-Firmware auf das Laufwerk "RPI-RP2" per Drag und Drop kopieren.
 4. Pico startet neu
- Danach kann der Pi Pico immer wieder eingesteckt und Thonny geöffnet werden
- Der Pi Pico verbindet sich dann automatisch mit Thonny
- main.py auswählen und "Run current script" Button klicken um zu starten
- Bei Spannungsversorgung ohne Datenfluss (zum Beispiel Pico direkt über USB-Kabel an Netzteil) startet main.py automatisch

Gehäuse

- Das Gehäuse besteht aus mehreren Komponenten, die in "documentation/technical drawings" und auch den "documentation/stl-files" gefunden werden können
- Der Peltier-Adapter kann an die Rückseite des Gehäuses eingeschoben werden
- Weitere Komponenten werden verschraubt
- Die stl-files sind 3D-Modell-Dateien, die in einem beliebigen CAD-Programm geöffnet werden können
- Die Technischen Zeichnungen und der Schaltplan sind als SVG-Vektorgrafiken gespeichert
- In CAD-file findet sich die FreeCAD 1.0.2 Projektdatei, welche alle Modelle, Technische Zeichnungen und deren Entwicklungsverlauf aufzeigt
- Bilder des Aufbaus finden sich im Verzeichnis "pico exterior"

Backend server

- Verbinde dich über ssh mit dem Backend
- Verbinde dich mit `nmtui` mit dem WiFi:
 1. Führe `nmtui` aus
 2. Wähle "Activate a connection"
 3. Wähle das Netzwerk aus und gebe das Passwort ein (Standard: Production, Production-01)
- Kclone das Repository auf das Backend
- Installiere Docker Compose
- Zum Starten der Software führe jetzt `sudo docker compose up` im Projektverzeichnis aus

Bedienung

Die Applikation lässt sich mit `sudo docker compose up/down` starten und beenden. Vor dem Start ist sicherzustellen, dass der WLAN-AP Pi hochgefahren und funktionstüchtig ist (z.B. indem geprüft wird, ob das Backend 'mit dem zugehörigen WLAN verbunden ist). Der Pico wird durch Ausführen der `main.py` gestartet. Nach dem Start kann eine Verbindung zum Netzwerk aufgebaut werden und unter <https://local.kleber.data> das Web Dashboard besucht werden. Hier folgt eine Anmeldung mit `[iotuser01]: [password]` oder `[admin]: [admin]`. Danach wird das Dashboard angezeigt. Als Admin besteht außerdem die Möglichkeit alle Sensordaten als CSV-Datei zu exportieren.

E2E-tests Ergebnisse

Ausgeführt am 10.06.2026 auf dem Raspberry Pi Backend.

Test	Beschreibung	Ergebnis	Details
Test 1	Sensor → DB	PASS (4/4)	Temperatur 17.77 in DB gefunden
Test 2	Keycloak Token Flow	PASS (5/5)	Rolle controller-ingest vorhanden, Bearer akzeptiert
Test 3	No Auth Rejected	PASS (4/4)	Alle unautorisierten Requests korrekt abgelehnt
Test 4	Invalid Data Rejected	PASS (3/3)	Kein neuer DB-Eintrag nach 5 ungültigen Nachrichten
Test 5	Actuator Control	PASS (3/3)	15× COOL auf actuator/control empfangen
Test 6	TLS Validation	PASS (3/3)	CA-Zertifikat validiert, falsches Zertifikat abgelehnt
Test 7	Dashboard Access	PASS (8/8)	Admin: voll, User: eingeschränkt, Ohne Rolle: abgelehnt

****Gesamtergebnis: 7/7 Tests bestanden ****

Fazit

Das Fazit ist unterteilt in die Bewertung der Erfüllung der gegebenen Anforderungen und den persönlichen Erfahrungsgewinn des Teams.

Anforderungen

Funktionale Anforderungen:

Nr.	Funktion	Prio	Bewertung
FA 1	Sensormessung	Must	Erfüllt; Der Pico misst meistens realistische Werte, es wird aber vermutet, dass diese manchmal leicht zu hoch sind
FA 2	Datenübertragung	Must	Erfüllt; getestet in E2E Test 1
FA 3	Automatische Temperatursteuerung	Must	Erfüllt im Rahmen der technischen Möglichkeiten; Temperatur kann erhöht und reduziert werden; Sollwerte sind aber zu weit von Außentemperatur entfernt, um diese zu erreichen; Es entstand das Gefühl, dass zum einen der Temperatursensor nicht immer korrekt misst; Die Luftzirkulation in unserem Gehäuse ist auch nicht optimiert
FA 4	Automatische Feuchtigkeitssteuerung	Must	Erfüllt im Rahmen der technischen Möglichkeiten; Luftfeuchtigkeit kann erhöht und reduziert werden; Programm ist mit der Temperaturanpassung stark ausgelastet; Luftfeuchtigkeit wird erst danach reguliert;
FA 5	Web-Dashboard (Visualisierung)	Must	Erfüllt; Visualisiert Sensordaten der letzten Woche; Demonstration in Life-Demo
FA 6	Manueller Eingriff	Must	Nicht erfüllt; Ausschließlich automatische Steuerung durch controller; Aus Sicherheitsgründen keine Kommunikation zwischen Pico und Webserver
FA 7	Datenspeicherung Kurzzeit	Should	Erfüllt; Sensordaten werden eine Woche in einer Tabelle gespeichert und danach archiviert
FA 8	Datenspeicherung Langzeit	Should	Erfüllt; Sensordaten werden nach einer Woche archiviert; Sind danach im Archiv gespeichert

Das Produkt erfüllt die wichtigsten funktionalen Anforderungen.

Es fehlt aber der manuelle Eingriff vom Dashboard.

Dieser wäre relevant, wenn das LSTM-Modell es nicht schafft die richtigen Steuersignale an den Pico zu schicken.

Zur Kontrolle wurde in E2E Test 5 geprüft, ob der Controller bei kritisch hohen Temperaturen auf jeden Fall kühlt.

Außerdem gehen wir davon aus, dass der verwendete Temperatursensor defekt ist.

Deswegen ist unklar, inwiefern der gewünschte Temperaturbereich reguliert werden kann.

Nicht-funktionale Anforderungen:

Nr.	Anforderung	Prio	Bewertung
NFA 1	Sicherheit – Verschlüsselung	Must	Erfüllt; Alle Datentransporte sind durch selbstsignierte Zertifikate gesichert; getestet in E2E Test 6
NFA 2	Authentifizierung & Autorisierung	Must	Erfüllt; Datenabfragen getestet mit E2E Test 7; Kein Zugriff auf Dashboard ohne gültige Anmeldung
NFA 3	Netzwerkisolation	Must	Erfüllt; Sendezugriff von außen durch nftables begrenzt auf mqtt Port 8883 und ssh Port 22
NFA 4	Verfügbarkeit / Zuverlässigkeit	Must	Nicht erfüllt; Aus Sicherheitsgründen keine Kommunikation zwischen Pico und Webserver
NFA 5	Kosteneffizienz	Must	Erfüllt; Verwendete Software ist open-source; Zusätzliche Hardware wie das Peltier-Element kostet nur wenige Euro und liegt somit im Budget-Rahmen
NFA 6	Wartbarkeit	Should	Erfüllt; Es wurden nur Standardisierte Bibliotheken und Docker Images verwendet
NFA 7	Mobilität / WLAN	Must	Erfüllt; Kommunikation über eigenes Production-Wlan
NFA 8	Datenschutz (DSGVO)	Should	Erfüllt; Keine kritischen Daten; Sicherheit wird eingehalten
NFA 9	Benutzbarkeit	Could	Nach eigenem Ermessen erfüllt; Das Dashboard wird als Nutzerfreundlich genug eingeschätzt, um von IT-Fremden Personal bedient zu werden

Die wichtigsten Nicht-Funktionalen Anforderungen sind erfüllt. In Bezug auf die Zuverlässigkeit wurde sich dafür entschieden aus Sicherheitsgründen auf manuelle Steuerung zu verzichten.

Erfahrungsgewinn

Das Projekt war nicht nur kompliziert, sondern wies eine hohe Komplexität auf, die durch die unterschiedlichen Qualifikationen der Entwickler zusätzlich verstärkt wurde. Diese ermöglicht einen großen Erfahrungsgewinn.

Fachliche Kompetenz

Im Laufe des Projekts wurden die bestehenden Kompetenzen zu Docker vertieft. Gerade in Bezug auf Sicherheit wurde Folgendes erlernt:

1. Container in Netzwerken strukturieren, damit sie nicht nach außen sichtbar sind
2. Schnittstellen nach außen gezielt auswählen (Ports, http/https ...)
3. An den Außenschnittstellen Sicherheit gewährleisten (nginx schützt durch OWASP/modsecurity)
4. Validierung von externen Daten nach jeder Kommunikation

Auch in Bezug auf CIA - Confidentiality, Integrity und Availability wurde einiges gelernt:

1. Nur authentifizierte Nutzer (bei uns der controller) können Daten an die Datenbank weitergeben (Integrity)
2. Keycloak bietet eine einfache und unkomplizierte Methode zur Authentifizierung (Availability)
3. Nutzer sind je nach Account zu unterschiedlichen Operationen berechtigt.
Nur mit den richtigen Rechten dürfen Daten abgefragt werden (confidentiality)

In Bezug auf Sicherheit wurden uns auch mögliche Sicherheitslücken gezeigt. Hierzu gehören Cross-Site-Scripting, SQL-Injection und Directory-Traversal.

Wechsel der Projektmanagement-Plattform

Während der Projektentwicklung musste die Projektmanagement-Plattform mehrfach gewechselt werden, da es wiederholt zu Zugangssperren kam. Ursache waren nicht offensichtlich gekennzeichnete Probezeiträume: Nach deren Ablauf wurden entweder kostenpflichtige Beiträge gefordert, oder der Zugriff auf die bereits eingegebenen Daten ging verloren.

Dieses Problem lässt sich vermeiden, indem die AGB vor der Nutzung genau gelesen oder von vornherein selbst gehostete Management-Tools eingesetzt werden.

Unklare Abhängigkeiten, Aufwände und Kompetenzen

Es ergaben sich Unklarheiten bei der Einteilung der Unteraufgaben und der Planung ihrer Abhängigkeiten.

Als Reaktion darauf wurde ein Projektstrukturplan (PSP) adaptiv eingesetzt; seine Aufgabenzuordnung diente als initiale Vorgehensweise für die jeweilige Projektphase.

Ergänzend wurde eine Verantwortungszuordnung genutzt, um mehrere Phasen gleichzeitig an verschiedenen Stellen zu starten.

Während der Bearbeitung traten jedoch latente Abhängigkeiten zutage, die zu vermeidbaren Verzögerungen führten.

Um diese Effekte zu minimieren, wurde mit einem agilen PDCA-Zyklus gearbeitet: Änderungen wurden direkt im Main-Branch umgesetzt und unmittelbar auf ihre Kompatibilität getestet.

Statt autoritärer Entscheidungsfindung erfolgten möglichst direkte Absprachen.

Erleichtert wurde dies durch die Git-Autorenverfolgung, da sich darüber nachvollziehen ließ, welcher Entwickler an welchem Codeabschnitt gearbeitet hatte und über welche Kompetenzen er verfügte.

Eine Folge dieses agilen Vorgehens war eine ungleiche Verteilung der Arbeitsaufwände. Gelöst wurde dies durch Kanban-Elemente wie „To-do“, „In Bearbeitung“, „Zu testen“ und „Fertig“, wodurch offene Aufgaben gezielt gefunden und übernommen werden konnten.

Weitere Hürden im Projektverlauf

Das Zeitmanagement der Vorlesungen, Selbststudium und Projektarbeiten zeigte klare Konflikte bei der Bearbeitung des Projekts, es musste flexibel geplant werden, da andere Tätigkeiten meist prioritär waren und somit das Abschließen von Projektaufgaben sich verzögerte.

Weiterhin bestanden Engpässe in den Lieferzeiten verwendeter, zusätzlicher Komponenten wie zum Beispiel dem Peltier-Element und den dazugehörigen Netzteilen.

Viele Aufgaben einzelner Phasen konnten nicht innerhalb der jeweiligen Phasen abgeschlossen werden, weshalb diese in den nächsten Phasen weiter bearbeitet wurden.

Literatur und Hilfsmittel

<https://projects.raspberrypi.org/en/projects/getting-started-with-the-pico/3>

Zugriff: 18.04.2026

Pico W firmware

<https://pip-assets.raspberrypi.com/categories/686-raspberry-pi-pico-w/documents/RP-008312-DS-1-pico-w-datasheet.pdf?disposition=inline>

Zugriff: 18.04.2026

Pico W datasheet

<https://docs.micropython.org/en/latest/rp2/quickref.html>

Zugriff: 24.04.2026

RP2 code reference

https://github.com/micropython/micropython-lib/blob/master/micropython/umqtt.simple/example_sub.py

Zugriff: 18.05.2026

umqtt reference

<https://www.drawio.com/>

Zugriff 10.06.2026

Deployment-Diagramme und Schaltplan

FreeCAD 1.0.2: Erstellung der 3D-Modelle und technischen Zeichnungen

Inkscape: Erstellung LOGO.svg

Bambu Studio: 3D-Modelle für den Druck slicen

Bambu Lab P1S-Drucker: Drucken der 3D-Modelle

Google Übersetzer: Deutsch zu Englisch

<https://claude.ai/>

Claude Code: Eingesetzt bei Recherche, Code-Vorschlägen, zusammenfassen/erklären und Dokumentation.

Beispiel-Prompt: Übersetze diese deutschen Kommentare und Docstrings in meinen Code ins Englische, ohne die Logik zu verändern

Beispiel-Prompt: Warum schlägt mein E2E-Test fehl, obwohl die Daten in der Datenbank ankommen?

Beispiel-Prompt: Erkläre mir was genau ich brauche um einen Controller zu bauen.

<https://gemini.google.com/app>

Google Gemini: Eingesetzt zur Fehlercode-Analyse.

Beispiel-Prompt: What is the source of the following error on python3.13 if it works on a python3.12-slim: "ca cert does not include key usage extension"

<https://copilot.microsoft.com/>

Microsoft Copilot: Eingesetzt für Fehleranalyse und Informationsrecherche.

Beispiel-Prompt: Mir wird aktuell dieser Fehlercode ausgegeben, könntest du mir erläutern, wieso dieser geworfen wird?

Beispiel-Prompt: Finde mir Datenblätter zu diesem Bauteil.

<https://github.com/>

Code-Repository